

Curs 3: Implementarea funcțiilor binare cu rețele de porți logice

Cuprins:

- **Prezentarea sistemelor complete de operatori logici;**
- **Implementarea funcțiilor binare cu rețele de porți logice în logică combinată;**
- **Implementarea funcțiilor binare în logică de tip NAND-NAND;**
- **Implementarea funcțiilor binare în logică de tip NOR-NOR**

1. Sisteme complete de operatori

Un **sistem complet de operatori** reprezintă un ansamblu, format din unul sau mai mulți operatori logici, cu ajutorul cărora se poate descrie cu succes orice funcție logică, indiferent de complexitatea acesteia.

Exemple de sisteme complete de operatori:

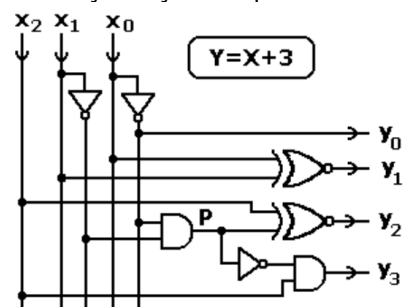
- $[com, min, max] \Leftrightarrow [NOT, AND, OR];$
- $[com, min] \Leftrightarrow [NOT, AND];$
- $[com, max] \Leftrightarrow [NOT, OR];$
- $[comin] \Leftrightarrow [NAND];$
- $[comax] \Leftrightarrow [NOR];$

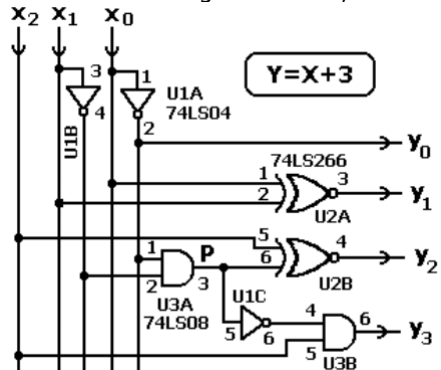
Trebuie remarcat că, la transpunerea în practică a unei scheme logice, fiecare operator trebuie înlocuit cu un circuit electronic specific. Deci, cu cât numărul operatorilor dintr-un sistem complet este mai redus, cu atât realizarea practică este mai ușoară deoarece presupune repetarea unui număr mai mic de circuite electronice distincte. Spre exemplu, o funcție logică poate fi realizată folosind sistemul $[NOT, AND, OR]$ - caz în care se repetă în mod convenabil trei tipuri de circuite electronice diferite, fie folosind sistemul $[NAND]$ - în care este necesară repetarea unui singur tip de circuit.

2. Implementarea funcțiilor binare în logică combinată

Etapele ce trebuie parcurse pentru implementarea unui CLC cu rețea de porți logice, în logică combinată, sunt prezentate în detaliu în tabelul de mai jos pentru un circuit logic care adună constanta 3 la numărul de intrare exprimat pe 3 biți.

Nr. etapă	Denumire etapă	Exemplu
1	Definirea foarte exactă a funcției logice ce trebuie realizate. Se pleacă de la descrierea în limbaj natural a funcționării circuitului.	Ce trebuie să facă circuitul ? - adună la numărul de intrare X , constanta zecimală 3 . - funcția de transfer a CLC-ului este: $Y = X + 3$. - starea logică a intrărilor este interpretată ca fiind scrierea binară a numărului de intrare X , iar starea logică a ieșirilor se consideră a fi scrierea binară a numărului Y . Câte intrări și câte ieșiri are circuitul ? - din datele inițiale știm că CLC-ul are 3 intrări, $X(x_2, x_1, x_0)$. - numărul de ieșiri trebuie deduse ținând cont de funcția realizată de CLC. - cel mai mare număr exprimat pe 3 biți este $X_{max}=111$, ceea ce corespunde cifrei zecimale 7; - cel mai mare număr de la ieșirea CLC-ului este: $Y_{max} = X_{max} + 3$ $Y_{max} = 7 + 3 = 10$, exprimat în baza 10 $Y_{max} = 1010$, exprimat în baza 2 - deoarece exprimarea lui Y_{max} se face pe 4 biți tragem concluzia că circuitul trebuie să prezinte 4 ieșiri, $Y(y_3, y_2, y_1, y_0)$.
	Alegerea unei	Pentru acest circuit, cea mai adecvată metodă de reprezentare este dată de

2	modalități de descriere a funcției logice. În principiu, se poate alege orice metodă de reprezentare cunoscută.	tabelul de adevăr complet. <table><tr><th>X</th><th>x₂</th><th>x₁</th><th>x₀</th><th>y₃</th><th>y₂</th><th>y₁</th><th>y₀</th><th>Y</th></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td><td>1</td><td>3</td></tr><tr><td>1</td><td>0</td><td>0</td><td>1</td><td>0</td><td>1</td><td>0</td><td>0</td><td>4</td></tr><tr><td>2</td><td>0</td><td>1</td><td>0</td><td>0</td><td>1</td><td>0</td><td>1</td><td>5</td></tr><tr><td>3</td><td>0</td><td>1</td><td>1</td><td>0</td><td>1</td><td>1</td><td>0</td><td>6</td></tr><tr><td>4</td><td>1</td><td>0</td><td>0</td><td>0</td><td>1</td><td>1</td><td>1</td><td>7</td></tr><tr><td>5</td><td>1</td><td>0</td><td>1</td><td>1</td><td>0</td><td>0</td><td>0</td><td>8</td></tr><tr><td>6</td><td>1</td><td>1</td><td>0</td><td>1</td><td>0</td><td>0</td><td>1</td><td>9</td></tr><tr><td>7</td><td>1</td><td>1</td><td>1</td><td>1</td><td>0</td><td>1</td><td>0</td><td>10</td></tr></table>	X	x ₂	x ₁	x ₀	y ₃	y ₂	y ₁	y ₀	Y	0	0	0	0	0	0	1	1	3	1	0	0	1	0	1	0	0	4	2	0	1	0	0	1	0	1	5	3	0	1	1	0	1	1	0	6	4	1	0	0	0	1	1	1	7	5	1	0	1	1	0	0	0	8	6	1	1	0	1	0	0	1	9	7	1	1	1	1	0	1	0	10
X	x ₂	x ₁	x ₀	y ₃	y ₂	y ₁	y ₀	Y																																																																											
0	0	0	0	0	0	1	1	3																																																																											
1	0	0	1	0	1	0	0	4																																																																											
2	0	1	0	0	1	0	1	5																																																																											
3	0	1	1	0	1	1	0	6																																																																											
4	1	0	0	0	1	1	1	7																																																																											
5	1	0	1	1	0	0	0	8																																																																											
6	1	1	0	1	0	0	1	9																																																																											
7	1	1	1	1	0	1	0	10																																																																											
3	Alegerea modalității (sau a tehnologiei) de implementare.	În această etapă trebuie avute în vedere aspecte precum: - performanțele propuse (viteză de lucru, consum de energie, etc.); - costul circuitelor utilizate; - numărul de exemplare în care se va produce circuitul, etc. În cazul de față, prin datele inițiale de proiectare, optăm pentru utilizarea porților logice.																																																																																	
4	Simplificarea funcției logice. Această etapă nu este strict necesară pentru toate modalitățile de implementare a funcțiilor logice. Pentru implementarea cu porți, această etapă este necesară deoarece reduce numărul de circuite integrate. Proprietățile algebrei binare trebuie aplicate astfel încât să obține expresii finale cât mai simple posibil.	Avem de implementat 4 funcții binare ce depind de aceleași variabile de intrare. ♦ Ieșirea y₀. Din analiza tabelului de adevăr se observă că avem $y_0 = \bar{x}_0$ ♦ Ieșirea y₁. Folosind prima formă canonică obținem: $y_1 = \begin{vmatrix} \bar{x}_2 \bar{x}_1 \bar{x}_0 \\ \bar{x}_2 x_1 x_0 \\ x_2 \bar{x}_1 \bar{x}_0 \\ x_2 x_1 x_0 \end{vmatrix} = \begin{vmatrix} \bar{x}_2 \bar{x}_1 \bar{x}_0 \\ x_1 x_0 \\ \bar{x}_1 \bar{x}_0 \\ x_1 x_0 \end{vmatrix} = \begin{vmatrix} \bar{x}_2 \bar{x}_1 \bar{x}_0 \\ x_1 x_0 \end{vmatrix} = \begin{vmatrix} \bar{x}_1 \bar{x}_0 \\ x_1 x_0 \end{vmatrix} = XNOR(x_1, x_0)$ ♦ Ieșirea y₂. Folosind prima formă canonică obținem: $y_2 = \begin{vmatrix} \bar{x}_2 \bar{x}_1 x_0 \\ \bar{x}_2 x_1 \bar{x}_0 \\ \bar{x}_2 x_1 x_0 \\ x_2 \bar{x}_1 \bar{x}_0 \end{vmatrix} = \begin{vmatrix} \bar{x}_2 x_0 \\ x_1 \bar{x}_0 \\ x_1 x_0 \\ x_2 \bar{x}_1 \bar{x}_0 \end{vmatrix} = \begin{vmatrix} \bar{x}_2 x_0 \\ x_1 \bar{x}_0 \\ x_1 x_0 \end{vmatrix} = \begin{vmatrix} \bar{x}_2 \bar{P} \\ x_2 P \end{vmatrix} = XNOR(x_2, P);$ unde $P = \bar{x}_1 \bar{x}_0$ ♦ Ieșirea y₃. Folosind prima formă canonică obținem: $y_3 = \begin{vmatrix} x_2 \bar{x}_1 x_0 \\ x_2 x_1 \bar{x}_0 \\ x_2 x_1 x_0 \end{vmatrix} = x_2 \begin{vmatrix} \bar{x}_1 x_0 \\ x_1 \bar{x}_0 \\ x_1 x_0 \end{vmatrix} = x_2 \begin{vmatrix} x_1 \\ x_0 \end{vmatrix} = x_2 \bar{P}$ Observații: - Funcția y₁, se implementează cu o poartă sau XNOR; - Produsul P, apare în expresia a două funcții binare, el va fi calculat o singură dată și apoi folosit în sinteza ambelor funcții;																																																																																	
5	Deducerea schemei logice. Atenție: Există o mulțime de scheme logice pentru același tabel de adevăr! Schema logică depinde foarte mult de modul în care s-au aplicat proprietățile algebrei binare în etapa de simplificare a	Schema logică rezultă din relațiile obținute la punctul anterior.  Se observă că această implementare necesită 7 porți logice dar acestea se găsesc																																																																																	

	expresiei algebrice.	În 3 circuite integrate diferite.
6	Implementarea hardware. După alegerea familiei logice trebuie identificate codurile circuitelor ce conțin porțile implicate în schema logică, apoi se face alocarea pinilor. După terminarea acestei etape avem toate datele necesare pentru realizarea practică a circuitului.	Alegem familia 74LS***. Consultând un catalog de firmă găsim următoarele coduri: - pentru inversoare: 74LS04; - pentru poarta AND cu două intrări: 74LS08; - pentru poarta sau exclusiv negat: 74LS266;  Se observă că am avut nevoie de 3 circuite integrate însă gradul lor de utilizare este următorul: 1/2 din 74LS04, 1/2 din 74LS266 și 1/2 din 74LS08

3. Implementarea funcțiilor binare în logică NAND-NAND

Logica de același tip presupune folosirea unui singur operator pentru implementarea întregului circuit logic combinațional. În mod curent sunt foarte utilizate două tipuri de logică de același tip:

- Logica de tip NAND-NAND care folosește doar porți de tip ȘI-NU;
- Logica de tip NOR-NOR care folosește doar porți de tip SAU-NU;

Mai există și alte tipuri de logică de același fel, spre exemplu bazată doar pe porți XOR, dar sunt mai rar folosite.

Față de etapele prezentate anterior, implementarea în logica de același tip necesită mici modificări în etapele prezentate anterior. În principal, după extragerea și simplificarea expresiilor algebrice se aplică în mod convenabil teoremele lui DeMorgan în scopul eliminării tuturor operațiilor de adunare.

În continuare prezentăm modul de implementare cu porți NAND a circuitului din exemplul anterior (circuitul de adunare cu 3 a numărului binare intrare).

Se pornește de la expresiile algebrice simplificate obținute în problema anterioară, după care se aplică în mod convenabil teoremele lui DeMorgan. Aceste teoreme au ca scop eliminarea sumelor logice din expresiile inițiale.

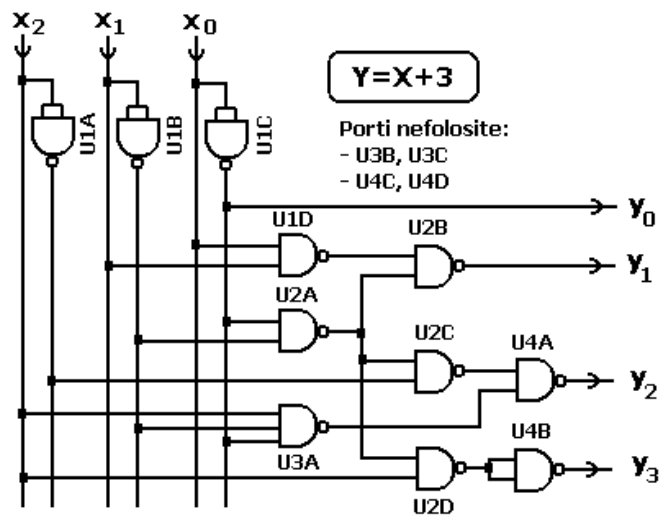
$$y_0 = \bar{x}_0$$

$$y_1 = \left| \begin{array}{c} \bar{x}_1 \quad \bar{x}_0 \\ x_1 \quad x_0 \end{array} \right| = \overline{\bar{x}_1 \bar{x}_0} \quad \overline{x_1 x_0}$$

$$y_2 = \left| \begin{array}{c} \bar{x}_2 \quad \left| \begin{array}{c} x_0 \\ x_1 \end{array} \right| \\ x_2 \quad \bar{x}_1 \bar{x}_0 \end{array} \right| = \left| \begin{array}{c} \bar{x}_2 \quad \overline{\bar{x}_1 \bar{x}_0} \\ x_2 \quad \bar{x}_1 \bar{x}_0 \end{array} \right| = \overline{\bar{x}_2 \quad \bar{x}_1 \bar{x}_0} \quad \overline{x_2 \quad \bar{x}_1 \bar{x}_0}$$

$$y_3 = x_2 \left| \begin{array}{c} x_1 \\ x_0 \end{array} \right| = x_2 \left| \begin{array}{c} x_1 \\ x_0 \end{array} \right| = x_2 \quad \overline{\bar{x}_1 \bar{x}_0} = x_2 \quad \overline{\bar{x}_1 \bar{x}_0}$$

Din aceste relații rezultă schema de mai jos:



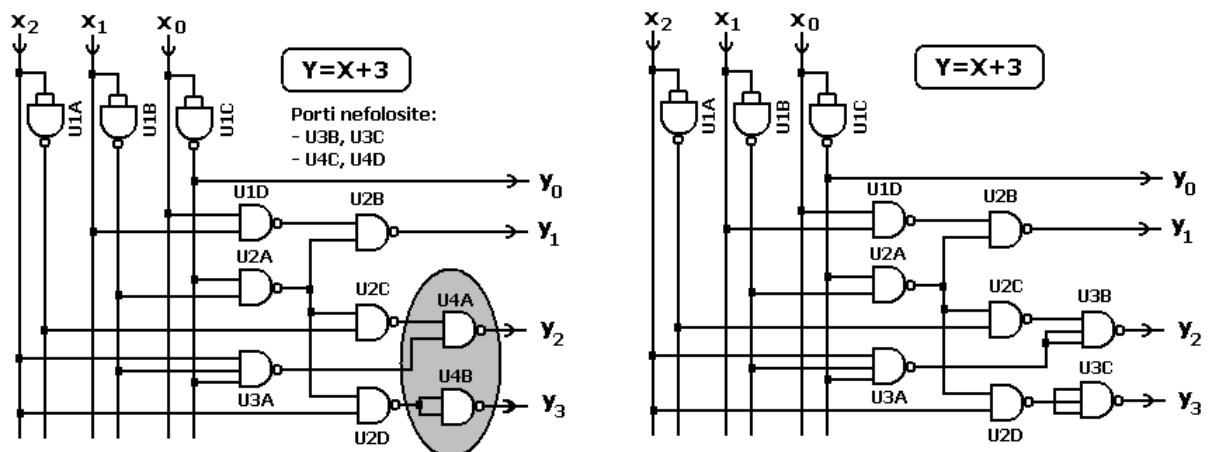
Circuitul $Y=X+3$, implementat în logică NAND-NAND

Această realizare necesită 10 porți NAND cu 2 intrări și o poartă NAND cu 3 intrări. Așadar, avem nevoie de 4 circuite integrate: 3 circuite 74LS00 și un circuit de tip 74LS10.

Analizând schema anterioară se poate constata că nu avem o utilizare eficientă a circuitelor integrate: din U4 folosim doar 2 porți din cele 4 disponibile, iar din circuitul U3 folosim doar o poartă din cele 3 disponibile.

O variantă de implementare cu un număr mai mic de circuite integrate se poate obține dacă cele două porți nefolosite din U3 (de tip NAND cu 3 intrări) vor fi configurate ca porți NAND cu 2 intrări și vor fi folosite în locul porților din U4. Prin acest artificiu nu mai avem nevoie de circuitul U4, schema va fi realizată doar cu 3 circuite integrate, cu utilizate completă.

Trecerea de la schema cu 4 circuite integrate la cea cu 3 este prezentată în figura de mai jos.



Reducerea numărului de capsule de circuit integrat

3. Implementarea funcțiilor binare în logică NOR-NOR

Pentru a obține o implementare de tip NOR-NOR teoremele lui DeMorgan sunt aplicate în scopul eliminării operațiilor de înmulțire din expresiile algebrice ale funcțiilor de ieșire.

În continuare prezentăm modul de implementare cu porți NOR a circuitului din exemplul anterior (circuitul de adunare cu 3 a numărului binare intrare).

Am arătat anterior că expresiile algebrice pentru acest circuit sunt:

$$y_0 = \bar{x}_0$$

$$y_1 = \left| \begin{array}{c} \bar{x}_1 \quad \bar{x}_0 \\ x_1 \quad x_0 \end{array} \right|$$

$$y_2 = \left| \begin{array}{c} \bar{x}_2 \quad x_0 \\ x_2 \quad \bar{x}_1 \end{array} \right|$$

$$y_3 = x_2 \left| \begin{array}{c} x_1 \\ x_0 \end{array} \right|$$

Pentru o scriere mai simplă, de această dată, pentru sumele logice vom utiliza notația clasică de forma: $a+b$.

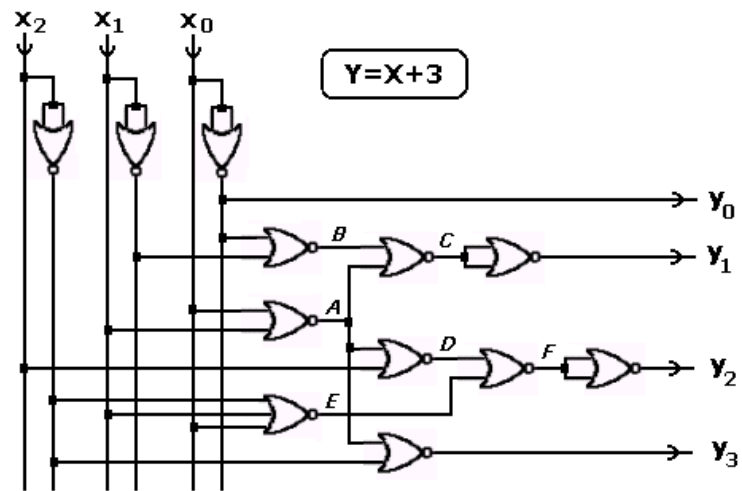
$$y_0 = \bar{x}_0 = NOR(x_0, x_0)$$

$$\begin{aligned} y_1 &= \left| \begin{array}{c} \bar{x}_1 \quad \bar{x}_0 \\ x_1 \quad x_0 \end{array} \right| = \bar{x}_1 \bar{x}_0 + x_1 x_0 = \overline{(x_1 + x_0)} + \overline{(\bar{x}_1 + \bar{x}_0)} = NOR(x_1, x_0) + NOR(\bar{x}_1, \bar{x}_0) = A + B = \\ &= \overline{A + B} = \overline{NOR(A, B)} = \bar{C} = NOR(C, C) \end{aligned}$$

$$\begin{aligned} y_2 &= \left| \begin{array}{c} \bar{x}_2 \quad x_0 \\ x_2 \quad \bar{x}_1 \end{array} \right| \stackrel{\text{DeMorgan}}{=} \overline{\bar{x}_2 (x_0 + x_1)} + \overline{x_2 \bar{x}_1 \bar{x}_0} \stackrel{\text{DeMorgan}}{=} \overline{\bar{x}_2 + (x_0 + x_1)} + \overline{\bar{x}_2 + x_1 + x_0} = \\ &= \overline{(x_2 + (x_0 + x_1))} + \overline{(x_2 + x_1 + x_0)} = \\ &= \overline{(x_2 + NOR(x_0, x_1))} + \overline{(x_2 + x_1 + x_0)} = \overline{(x_2 + A)} + \overline{(x_2 + x_1 + x_0)} = \\ &= NOR(x_2, A) + NOR(\bar{x}_2, x_1, x_0) = D + E = \overline{D + E} = \overline{NOR(D, E)} = \bar{F} = NOR(F, F) \end{aligned}$$

$$y_3 = x_2 \left| \begin{array}{c} x_1 \\ x_0 \end{array} \right| = x_2 (x_1 + x_0) = \overline{\bar{x}_2 + \overline{(x_1 + x_0)}} = NOR(\bar{x}_2, NOR(x_1, x_0)) = NOR(\bar{x}_2, A)$$

Din aceste relații rezultă schema de mai jos:

Circuitul $Y=X+3$, implementat în logică NOR-NOR**Obs:**

- Această realizare necesită 11 porți NOR cu 2 intrări și o poartă NOR cu 3 intrări. Așadar, avem nevoie de 4 circuite integrate: 3 circuite 7402 și un circuit de tip 7427;
- Față de implementarea cu porți NAND, varianta cu NOR folosește o poartă în plus;